DEBRE BERHAN UNIVERSITY

COLLEGE OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE

INTERNSHIP PROJECT PROPOSAL

Project title: Smart Cafeteria Ordering System

Submitted To: Internship Supervisors

Submitted By: Kidus Birhanu (Arbaminch University), Sintayehu Begashaw(Dredewa University) Wondesen Gemechu (Debre Tabor University) (3rd Year Computer Science Students.)

Submission Date: 29-11-2018 E.C (August 5, 2026)

Submitted for: review and approval

Table of Contents

1. Introduction

During our internship across several Ethiopian higher education institutions, we conducted operational observations of campus dining and commercial cafeteria services. We observed that the current food ordering, payment processing, and meal collection workflow is predominantly manual. Customers—comprising students, faculty, administrative staff, and visitors—are required to physically visit the cafeteria, queue in long lines to view paper menus, complete separate manual payment transactions via TeleBirr or CBE Birr receipts, and wait at service counters while food is prepared.
This traditional manual workflow results in cumulative waiting times ranging from 30 to 60 minutes per meal during peak lunch and dinner hours. The resulting dining room overcrowding and long service delays frequently disrupt academic schedules and administrative workflows. To eliminate these operational bottlenecks, we propose the Digital Cafeteria Ordering System. This solution is a responsive full-stack web platform enabling all campus users to view live menus, place advance remote orders and select pickup simulate seamless multi-channel digital payments with persistent database records, receive real-time order notifications, and collect meals immediately upon preparation.

2. Problem Statement

The current cafeteria system has several limitations:

- Long queues during peak hours (5-30 Min)

- 30-60 minutes required for a complete meal process

- Overcrowded cafeterias

- Paper-based ordering errors

- Academic & Work Disruptions.

- Customer Dissatisfaction

- Separate manual payment process

- Difficult to manage all Cafeteria Service Process For Cafeteria Service Provider.

- delay orders and reduce service quality.

- Customers cannot easily check whether their order is being prepared or is ready for collection.

Core Problem: There is no digital platform for remote ordering or payment before arriving at the cafeteria.

3. Proposed Solution

The proposed system is a web-based cafeteria ordering platform with:

- Digital menu with food images, descriptions, and prices

- Remote ordering from anywhere on campus

- Simulated CBE/TeleBirr payment

- Real-time order status tracking and Notification

- Secure user authentication (JWT)

- Admin panel for menu and order management

- Kitchen Display & Order Management Panel: Dedicated kitchen staff dashboard for viewing incoming tickets, changing order states, and notifying customers

Innovation: Students can order in advance and collect meals once they are prepared.

4. In Scope and Out of Scope

This project will develop a responsive web application that enables users to register and log in, browse a digital cafeteria menu with food images and prices, manage a shopping cart, place orders, simulate payments using CBE or TeleBirr, track order status in real time, view order history, and allow administrators to manage menus and customer orders. The system will be deployed online using Netlify and Render.

• Order & Fulfillment Options: Shopping cart management, pre-orders, express self-pickup, and local campus delivery location selection.
• Payment Simulation & Database Persistence: Interactive, realistic checkout simulation supporting TeleBirr, CBE Birr, and Mobile Banking. Every simulated payment generates an encrypted transaction record stored in the MongoDB database.
• Real-Time Status Notifications: Live updates on customer dashboards and kitchen queue

displays as order status transitions through fulfillment stages.
• Kitchen Service Counter Dashboard: Specialized portal for kitchen staff to receive, process, fulfill, and mark orders as served.
• Reporting & Analytics Module: Visual revenue summaries, daily/weekly order counts, item sales performance, and peak ordering time distribution tables.
• Activity & Transaction Logging: Secure, append-only activity tracking for security auditing, administrative oversight, and financial reconciliation.
• Web Deployment: Staging and production deployment using Netlify for the frontend client and Render for the Node.js backend.

However, the project will not include real payment gateway integration, native mobile applications, physical food delivery services, integration with the university student management system, advanced analytics dashboards, multilingual support, real-time customer chat, or external food delivery platform integration.

5. Technology Stack

Layer| Technologies

Frontend| HTML5, CSS3, JavaScript

Backend| Node.js, Express.js

Database| MongoDB (Mongoose)

Authentication| JWT, bcrypt

Version Control| Git, GitHub

Deployment| Netlify, Render, MongoDB Atlas

AI Support| ChatGPT, GitHub Copilot, Claude

All selected technologies are: free, widely used, and suitable for academic projects.

6. Core Features

Student

- Register / Login

- Browse Menu

- Search Food

- View Food Details

- Add Items to Cart

- Update Cart

- Remove Cart Items

- Place Orders

- Simulate Payment

- Receive Notifications

- Track Order Status

- View Order History

Food Maker

- Login

- View Incoming Orders

- Update Order Status

- Mark Orders as Ready

- Manage Food Preparation

Admin

- Login

- Manage Users

- Manage Menu Items

- Manage Categories

- View Orders

- Monitor Payments

- Generate Reports

- Manage System Settings

6.1 System Use Case Diagram

The following UML Use Case Diagram illustrates the main interactions between the two primary actors of the system—Student and Cafeteria Staff (Admin)—and the core functionalities provided by the Digital University Cafeteria Ordering System**.**

It shows how students can register, log in, browse the menu, place orders, simulate payments, and track order status, while administrators manage menus, receive orders, update order status, and send readiness notifications.

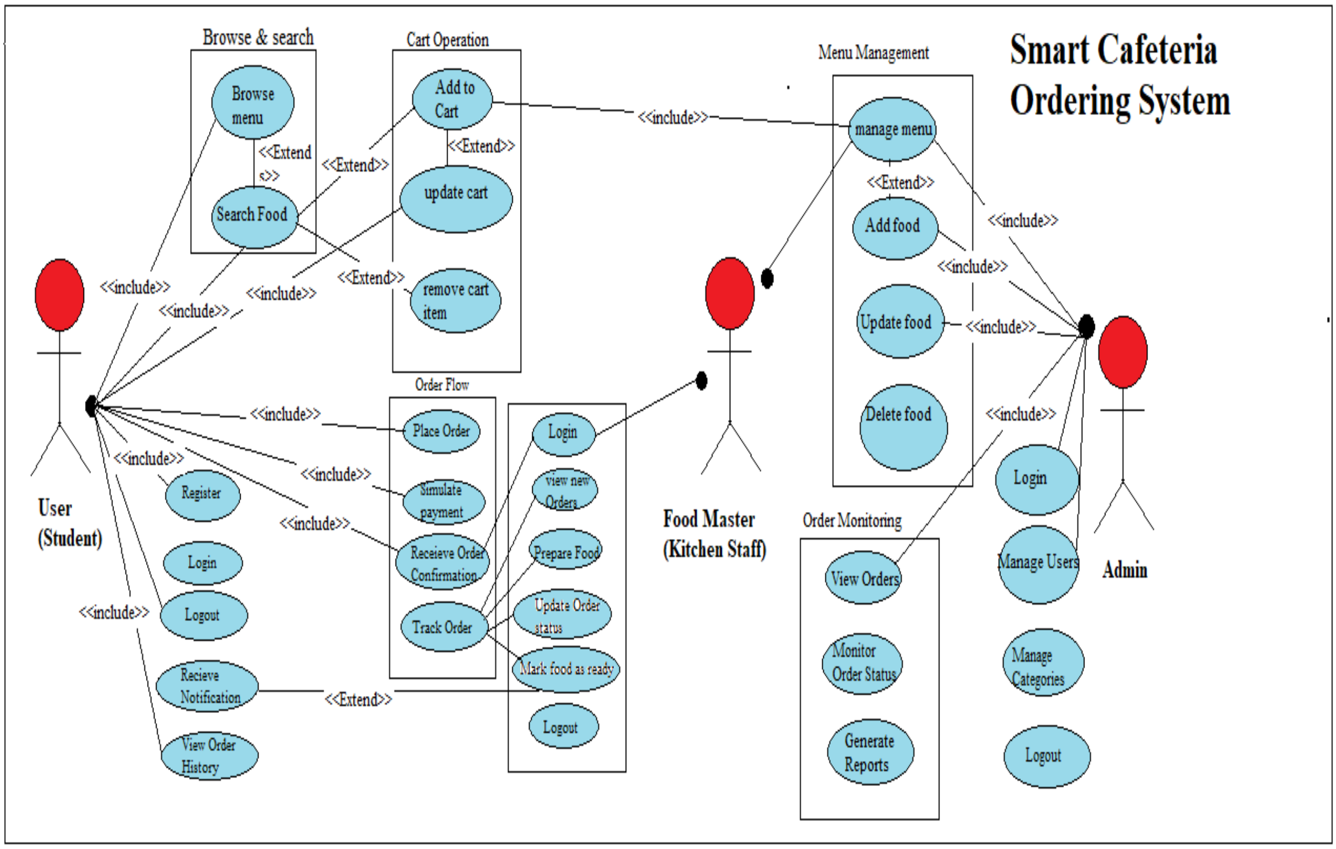

Figure 1.0: *Use Case Diagram Smart Cafeteria Ordering System*

Explanation: The Student actor interacts with the customer-side features such as registration, authentication, menu browsing, cart management, ordering, payment simulation, order tracking, and order history. The Cafeteria Staff (Admin) actor manages operational tasks including receiving orders, managing menu items, updating order status, and notifying students when meals are ready.

The Login use case includes Authenticate, indicating that user verification is required before accessing protected system features.

7. Reporting Module

The Reporting Module enables the Administrator to monitor and analyze the overall performance of the Smart Cafeteria Ordering System. It generates reports from system data to support decision-making and improve cafeteria operations. The module includes : -

Daily Order Report – Displays daily order statistics and order status.

Sales Report – Summarizes daily, weekly, and monthly sales.

Popular Food Report – Shows the most frequently ordered food items.

Customer Order Report – Displays customer order details and current order status.

Payment Report – Provides payment summaries, including successful, pending, and failed transactions.
These reports help administrators monitor system performance, improve food management, and support better operational decisions.

8. Project Objectives

Technical: Develop a responsive frontend, secure backend, MongoDB database, and deploy a fully functional web application using modern web technologies.

Functional: Enable remote ordering, payment simulation, order tracking, and administrative management.

Educational: Strengthen full-stack development skills, teamwork, and AI-assisted software development.

9. System Architecture

The Smart Cafeteria Ordering System follows a modern three-tier web architecture, consisting of the Presentation Layer, Application Layer, and Data Layer. This architecture separates the user interface, business logic, and data management into independent layers, making the system more secure, maintainable, scalable, and easier to manage.

Overall System Architecture Flow

Student (User)

Food Maker

Admin

Web Browser (Chrome, Edge, Firefox)

Frontend (HTML, CSS, JavaScript)
Hosted on Netlify

HTTP/HTTPS Requests

Backend Server (Node.js + Express.js)

Auth Menu Orders Users Payment Reports
Module Module Module Module Module Module

MongoDB Atlas Database

Stores user accounts, menu items, orders,
payment records, and report information

1. Presentation Layer (Frontend)Technologies:

HTML5

CSS3

JavaScript

- Architectural Purpose & Rationale: Provides lightweight, mobile-first responsive rendering for smartphones, tablets, and desktop workstations without heavy framework overhead.

The Presentation Layer provides the graphical user interface (GUI) through which Students, Food Makers, and Administrators interact with the system. Users can register, log in, browse the menu, search for food, manage their cart, place orders, simulate payments, receive notifications, track order status, and perform other system functions. All user requests are sent securely to the backend through REST APIs.

2. Application Layer (Backend) Technologies:

Node.js

Express.js

REST API

- Architectural Purpose & Rationale: Asynchronous, event-driven RESTful API backend handling concurrent client requests, business logic, and security middleware efficiently.

The Application Layer contains the core business logic of the system. It processes user requests, validates input data, manages authentication and authorization, handles menu management, processes customer orders, updates order status, generates notifications, and communicates with the MongoDB database through REST APIs.

3. Data Layer Technologies:

MongoDB Atlas

Mongoose ODM

The Data Layer is responsible for storing and managing all application data securely. It stores user accounts, food categories, menu items, shopping carts, customer orders, payment simulation records, notifications, and other system information. Mongoose ODM simplifies communication between the Node.js backend and the MongoDB Atlas database.

Authentication and Security

To ensure secure access and protect user information, the system implements the following security mechanisms:

JWT (JSON Web Token): Provides secure user authentication and session management after successful login.

bcrypt: Encrypts user passwords before storing them in the database, ensuring password confidentiality and protection against unauthorized access.

Deployment

The system will be deployed using modern cloud platforms:

Frontend: Netlify

Backend API: Render

Database: MongoDB Atlas

This deployment architecture enables secure, reliable, and scalable access to the Smart Cafeteria Ordering System over the internet.

10. Project Timeline (4 Weeks)

Week| Activities

The project will be completed within four weeks (Nehase 1–28, 2018 E.C. / August 2026). The team will work five days per week, dedicating approximately 4–6 hours per day to ensure the timely completion of all planned activities.

Week	Ethiopian Calendar (2018 E.C.)	Activities
Week 1	Nehase 1 – Nehase 7, 2018 E.C. (August 7–13, 2026)	Project planning, requirement analysis, UI/UX design, GitHub setup, database design
Week 2	Nehase 8 – Nehase 14, 2018 E.C. (August 14–20, 2026)	Frontend development, responsive interface, shopping cart, authentication pages
Week 3	Nehase 15 – Nehase 21, 2018	Backend development, REST APIs, MongoDB integration,

	E.C. (August 21–27, 2026)	JWT authentication, Admin Panel
Week 4	Nehase 22 – Nehase 28, 2018 E.C. (August 28–September 3, 2026)	System integration, testing, debugging, deployment, documentation, final presentation.

11. Project Budget Estimation

The following table presents the estimated expenses required to complete the Smart Cafeteria Ordering System project during the four-week development period. The budget covers transportation, internet access, documentation, and other operational costs incurred by the project team.

No.	Item	Quantity	Estimated Cost (ETB)
1.	Internet/Data Packages (approximately 15–25 ETB per day)	1 month	1000
2.	Transportation (30–100 ETB per trip, 3–5 days per week)	-	1,500
3.	Printing and Photocopying (Proposal, Report, and Presentation Materials)	2	400
4.	Stationery (Notebooks, Pens, etc.)	3	200
5.	Communication (Phone Calls and Mobile Data)	-	300

6.	Cloud Services and Software Tools (if required)	3	800
7.	Miscellaneous and Contingency Expenses	1	500
	Total Estimated Budget		**5,000 ETB**

Budget Description

Internet/Data: Used for online research, development, collaboration, GitHub access, testing, and deployment throughout the project.

Transportation: Covers travel to meetings, campus discussions, consultations with supervisors, and project-related activities.

Printing and Photocopying: Includes printing the proposal, final report, presentation materials, and other required documents.

Stationery: Covers notebooks, pens, and other office supplies used during the project.

Communication: Includes phone calls and mobile data for communication among team members and supervisors.

Cloud Services and Software Tools: Covers any optional online services or tools used during development and deployment.

Miscellaneous: Reserved for unexpected project-related expenses.

12. Team Composition and Responsibilities

The project is developed by a team of three Computer Science students, with each member assigned specific responsibilities based on their skills and area of focus. This division of tasks promotes effective collaboration, improves productivity, and ensures the successful completion of the Smart Cafeteria Ordering System within the project schedule.Member| Primary Responsibility : -

Kidus Birhanu	Frontend Development, UI/UX Design, Responsive Interface, Testing
Sintayehu Begashaw	Backend Development, REST API Development, Authentication, Integration
Wondesen Gemechu	MongoDB Database Design & Management, Deployment, Documentation, System Maintenance

Shared Responsibilities

➢Requirement Analysis

➢Project Planning

➢Problem Solving

➢Code Review

➢System Testing

➢Final Presentation

Supervisor Role: The project supervisor provides technical guidance, reviews the team's progress and code quality, assists in troubleshooting challenges, and evaluates the overall project to ensure it meets the required academic and technical standards.

13. Risk Management

Every software project may encounter technical, operational, or scheduling challenges during development. The following table identifies the major risks that may affect the Smart Cafeteria Ordering System and the strategies the team will use to minimize their impact.

Risk	Mitigation Strategy
Time Constraints	Prioritize core features and divide tasks among team members
Backend Challenges	Use AI assistance, official documentation, and team collaboration to resolve issues
Deployment Issues	Test the application early and deploy incrementally.
AI-Generated Code Errors	Review, test, and validate all AI-generated code before integration.
Security Risks	Implement JWT authentication, bcrypt password hashing, and input validation to protect the system.

14. Conclusion

The **Smart Cafeteria Ordering System** is designed to address common challenges in cafeteria services by reducing queues, minimizing waiting time, and providing a convenient online food ordering experience with payment simulation. The project also enables the team to apply practical full-stack web development skills using modern technologies. Through effective teamwork, proper planning, AI-assisted development, and a structured four-week implementation schedule, we are confident in delivering a secure, functional, and deployable web application that meets the project objectives.